Java Polymorphism

Section 1: Learn

What is Polymorphism?

Polymorphism means **"many forms"**. It allows the same method or object to behave differently based on context.

Java supports polymorphism to promote **flexibility**, **code reuse**, and **scalability**.

Types of Polymorphism

1. **Compile-time Polymorphism** (Method Overloading)
2. **Runtime Polymorphism** (Method Overriding)

Compile-Time Polymorphism (Method Overloading)

Multiple methods with the **same name but different parameters**.

```
class MathOps {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Key Points:

- Based on method signature (number/type of arguments)
- Resolved at **compile time** (Early Binding)

Runtime Polymorphism (Method Overriding)

Child class **redefines** a method of the parent class.

```
class Animal {
    void sound() { System.out.println("Animal sound"); }
}

class Dog extends Animal {
    void sound() { System.out.println("Bark"); }
}
```

Key Points:

- Decided during **runtime** (Late Binding)
- Enables **dynamic method dispatch**

Rules of Method Overriding

- Method name, return type, and parameters **must match**
- Access level **cannot be more restrictive**
- Only **inherited** methods can be overridden
- Constructors **cannot** be overridden
- Use @Override annotation for clarity

Is-A vs As-A Relationship

- **Is-A**: Inheritance (e.g., Dog is-an Animal)

- **As-A**: Usage (e.g., Animal behaves as-a Dog)

```
Animal a = new Dog();  // As-a Dog
```

Rules of Polymorphism with Interfaces

- Interface reference can point to any implementing class.
- You can call only the methods **declared in the interface**.

```
interface Printer {
   void print();
}

class Epson implements Printer {
   public void print() {
      System.out.println("Epson Printer");
   }
}

Printer p = new Epson(); // Valid
```

Real World Application

- **ATM Machine**: The withdraw() method behaves differently for SavingsAccount and CurrentAccount.
- **Payment Gateway**: pay() method works for UPI, Card, Wallet using polymorphism.

Early Binding vs Late Binding

- **Early Binding**: Resolved during **compile time** (e.g., method overloading)
- **Late Binding**: Resolved during **runtime** (e.g., method overriding)

Real World Example

```
class Employee {
    void work() {
        System.out.println("Working...");
    }
}

class Developer extends Employee {
    void work() {
        System.out.println("Writing code...");
    }
}

class Manager extends Employee {
    void work() {
        System.out.println("Managing team...");
    }
}

public class Company {
    public static void main(String[] args) {
```

```
        Employee e1 = new Developer(); // Late Binding
        Employee e2 = new Manager();

        e1.work(); // Writing code...
        e2.work(); // Managing team...
    }
}
```

Section 2: Practice

Try Overloading

```
class Calc {
    int multiply(int a, int b) { return a * b; }
    int multiply(int a, int b, int c) { return a * b * c; }
}
```

Try Overriding

```
class Bird {
    void fly() { System.out.println("Bird flies"); }
}

class Eagle extends Bird {
    void fly() { System.out.println("Eagle flies high"); }
}
```

Section 3: Know More (FAQs)

Can constructors be overloaded?

Yes, you can have multiple constructors with different parameters.

Can private methods be overridden?

No, private methods are not inherited, so they cannot be overridden.

What's dynamic method dispatch?

It's the mechanism that allows a call to an overridden method to be resolved at **runtime**.

Why is polymorphism important?

It helps write **flexible** and **extensible** code by allowing objects to take many forms.

Can we achieve polymorphism using interfaces?

Yes, interfaces are one of the best ways to implement polymorphism, especially in **large applications**.